

Vorlesung (WS 2016/17)

Softwarekonstruktion

Dr. Boris Düdder

Lehrstuhl XIV – Software Engineering
TU Dortmund, Fakultät Informatik

Teil 2.9: Algebraische Spezifikation

v. 16.01.2017

[inkl. Beiträge von Prof. Volker Gruhn und Dr. Johannes Henkel]

Literatur:

J. Ludewig, H. Lichter: Software Engineering - Grundlagen, Menschen, Prozesse, Techniken, dpunkt.verlag, 3. Auflage, 2013

https://www2.swc.rwth-aachen.de/se_buch . Ab 33 EUR (e-book).

Unibibliothek (2 Exemplare, 2. Auflage):

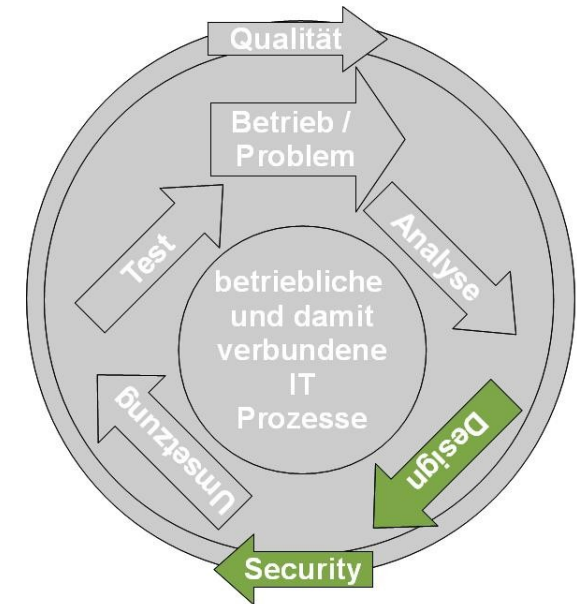
<http://www.ub.tu-dortmund.de/katalog/titel/1283989> .

(Bei Engpässen kann eine **Kopiervorlage** der relevanten Ausschnitte zur Verfügung gestellt werden.)

- **Kapitel 5**

- Erlernen und Anwenden von Methoden zur algebraischen Spezifikation von Software
- Kompetenz entwickeln beim Ableiten von Programmgarantien auf der Basis dieser Spezifikationen
- Entwickeln eines Verständnis der Probleme und Grenzen dieses Spezifikationsformalismus

- Modellgetriebene SW-Entwicklung
 - OCL
 - Modellbasierte Softwareentwicklung
 - EMF
- Qualitätsmanagement
- Softwareverifikation
 - Algebraische Spezifikation



2.9 Algebraische Spezifikation

2.9 Algebraische Spezifikation

Das Spezifikationsproblem

Signatur und Algebra

Vollständigkeit der Algebraischen Spezifikation

Algebraische Spezifikation: Diskussion

In diesem Kapitel: *Algebraische Spezifikation* als Grundlage für Spezifikation des Verhaltens einzelner Softwaremodule.

Warum ?

- **Verhalten einzelner Softwaremodule:** heutige Softwaresysteme sind von erheblicher Komplexität. → Durch Verwendung modularer Herangehensweisen beherrschbar.
- **Spezifikation:** Spezifikation Verhalten einzelner Softwaremodule wichtig.
→ Modulare Softwareentwicklung und Qualitätssicherung ermöglichen.
- **Grundlage:** Praxis: Verschiedene Ansätze zur Spezifikation des Verhalten einzelner Softwaremodule, z.B. UML, Java / C assertions, Java Markup Language (JML).
In diesem Abschnitt: Formale Grundlagen dazu betrachten.

Direkter Einsatz in einigen Anwendungen. (Beispiel Verifikation von kryptographischen Protokollen: Spezifikation kryptographischer Eigenschaften als algebraische Gleichungen; vgl. später).

- Betrachtet **abstrakte Datentypen** als algebraische Strukturen.
→ Mengen mit darauf definierten Operationen, für die bestimmte Axiome gelten.
- Dient zur **formalen Spezifikation des Verhaltens** von Modulen/Schnittstellen.
- Dient zur **Verifikation der Implementierung** gegen Spezifikation und Verifikation der Spezifikation gegenüber zu erfüllenden Anforderungen.
- Durch **Zusammensetzen solcher Module** kommt man zu **Software-Systemen**. → Definition deren Verhalten durch Spezifikation der Module und Konstruktion der Module zum Software-System.
- **Konstruktion von Software-Systemen** aus Modulen: Gegenstand des Entwurfs.

Nach [Som10] besteht Erstellungsprozess aus folgenden Schritten:

- 1) **Strukturieren der Spezifikation:** Alle Schnittstellen eines Systems identifizieren.
 - Miteinander kommunizierende Teile herausfinden
 - Notwendige Operationen informell beschreiben.
- 2) **Benennen der Spezifikation:** Alle abstrakten Datentypen einer Spezifikation mit Namen versehen (z.B. Liste) und entscheiden, ob sie generische Parameter benötigen.
- 3) **Auswahl der Operationen:** Bezeichnung aller Operationen festlegen.
- 4) **Informelles Spezifizieren der Operation:** Identifizierte Operationen sowie deren Auswirkungen auf System beschreiben.
- 5) **Definieren der Syntax:** Für jede identifizierte Operation Syntax formal beschreiben.
 - Alle Operationen zusammen mit ihren Parametern definieren. Syntax gesamter Spezifikation: Zusammensetzung aller abstrakten Datentypen.
- 6) **Definieren der Semantik:** Definition der Semantik der Operationen: Zutreffende Bedingungen (Axiome) für verschiedene Kombinationen von Operationen beschreiben.

[Som10] Sommerville, Ian: Software engineering, ch. 27: Formal Specification. Addison-Wesley Longman Publishing Co., Inc., 2010. http://www.cs.st-andrews.ac.uk/~ifs/Books/SE9/WebChapters/PDF/Ch_27_Formal_spec.pdf

Algebraische Spezifikation eines abstrakten Datentyps besteht aus Syntax- und Semantikteil:

- **Syntax:** Menge von Vereinbarungen von Zugriffsoperationen:
 - Operationsname: Definitionsbereich $\rightarrow$ Wertebereich.
- **Semantik:** Menge von Gleichungen, die Beziehungen zwischen Eingabe- und Rückgabewerten verschiedener Operationen in Form von Axiomen beschreiben.

Bemerkung: Algebraische Spezifikation bietet zunächst kein Konzept, interne Zustandsänderungen direkt zu beschreiben, sondern allenfalls indirekt, z.B. indem für jede Operation, für die Zustandsänderungen modelliert werden soll, ein Eingabe- und Rückgabewert vom Type „State“ (o.ä.) definiert wird (1). Es gibt allerdings Erweiterungen, die internen Zustand als Modellierungskonzept beinhalten (z.B. Abstract State Machines (2) oder Algebraic State Machines (3)).

(1) vgl. http://www.ldr.jaist.ac.jp/jaist-fssv2010/studentsSlides/zhang_slides.pdf ,
<http://www-plan.cs.colorado.edu/henkel/pubs.html>

(2) http://en.wikipedia.org/wiki/Abstract_state_machines

(3) <http://citeseerx.ist.psu.edu/viewdoc/download?doi=10.1.1.140.8498&rep=rep1&type=pdf>

algebra X

introduces sorts X,Y,Z

operations

$\text{op1} : X \rightarrow X$

$\text{op2} : X \rightarrow Y$

constraints op1, op2 so that for all

$x, y : X, z : Y$

...

Syntax

Semantik

algebra stack-of-nat **introduces** sorts nat, bool, stack-of-nat
operations

create: $\rightarrow$ stack-of-nat

push: stack-of-nat, nat $\rightarrow$ stack-of-nat

pop: stack-of-nat $\rightarrow$ stack-of-nat

top: stack-of-nat $\rightarrow$ nat

isempty: stack-of-nat $\rightarrow$ bool

verändernde
Operationen

inspizierende
Operationen

constraints create, push, pop, top, isempty
so that for all st: stack-of-nat, n: nat

isempty (create()) = true

isempty (push (st, n)) = false

pop (create()) = create()

pop (push (st, n)) = st

top (create()) = 0

top (push (st, n)) = n

Schritt 1) - 4):

- Erzeugen (*new*). Sorte *String* wird dafür gebraucht.
- Verketteten (*append*).
- Zeichen anhängen (*add*). Sorte *Char* wird außerdem gebraucht.
- Länge ermitteln (*length*). Sorte *Nat* wird außerdem gebraucht.

NB: Wir definieren hier die Sorte *Nat* als die natürlichen Zahlen inklusive 0.

- Ermitteln, ob leer (*isEmpty*). Sorte *Bool* wird außerdem gebraucht.
- Gleichheit zweier Zeichenketten (*equal*)

Schritt 4) hier implizit (es ist klar, was z.B. „verketteten“ für Auswirkungen hat).

algebra StringSpec

introduces sorts String, Char, Nat, Bool

operations

new: $\rightarrow$ String

append: String, String $\rightarrow$ String

add: String, Char $\rightarrow$ String

length: String $\rightarrow$ Nat

isEmpty: String $\rightarrow$ Bool

equal: String, String $\rightarrow$ Bool

Syntax

algebra StringSpec legt Syntax fest, aber was bedeuten Operationen?

- Beschreibung der **Semantik** der Operationen durch **Gleichungen**.
→ Formulieren Eigenschaften, die von Realisierung der Operationen nicht verletzt werden dürfen. Gleichungen heißen deshalb **Axiome**.
- Spätere Realisierungen der Operationen, die Axiome nicht verletzen: **spezifikationskonform** !
- „Lücken“ in Axiomen: Ursache für spätere Missverständnisse.
- (Informelle) **Definition**: Wenn sich nicht alle als wahr angenommenen Aussagen auch anhand Axiome ableiten lassen. → Algebraische Spezifikation **unvollständig**.

algebra StringSpec **introduces sorts** String, Char, Nat, Bool

operations

constraints new, append, add, length, isEmpty, equal

so that for all s,s1,s2: String, c: Char

$\text{isEmpty}(\text{new}()) = \text{true}$

$\text{isEmpty}(\text{add}(s,c)) = \text{false}$

$\text{length}(\text{new}()) = 0$

$\text{length}(\text{add}(s,c)) = \text{length}(s) + 1$

$\text{append}(s, \text{new}()) = s$

$\text{append}(s1, \text{add}(s2,c)) = \text{add}(\text{append}(s1,s2),c)$

$\text{equal}(\text{new}(), \text{new}()) = \text{true}$

$\text{equal}(\text{new}(), \text{add}(s,c)) = \text{false}$

$\text{equal}(\text{add}(s,c), \text{new}()) = \text{false}$

$\text{equal}(\text{add}(s1,c), \text{add}(s2,c)) = \text{equal}(s1,s2)$

Semantik

Frage: Sind diese Gleichungen (intuitiv) „korrekt“ ? Sind sie „vollständig“ ?

„Korrekt“ ? => Die o.g. Gleichungen sind intuitiv korrekt...

... allerdings nur, wenn bestimmte Annahmen an die Intuition erfüllt sind (die im Rahmen von Schritt 4 des Vorgehens bereits vorab informell spezifiziert worden sein sollten). Zum Beispiel:

- Das Axiom „ $\text{append}(s1, \text{add}(s2, c)) = \text{add}(\text{append}(s1, s2), c)$ “ setzt voraus, dass die intendierten Implementierungen von $\text{append}(s1, s2)$ und $\text{add}(s, c)$ konsistent bezüglich der Reihenfolgen von $s1$ und $s2$ bzw s und c sein sollen (z.B. $\text{append}(s1, s2) = [s1::s2]$ und $\text{add}(s, c) = [s::c]$ oder $\text{append}(s1, s2) = [s2::s1]$ und $\text{add}(s, c) = [c::s]$, aber nicht $\text{append}(s1, s2) = [s1::s2]$ und $\text{add}(c, s) = [c::s]$, wobei $[s1::s2]$ die Konkatenation von $s1$ gefolgt von $s2$ ist).
- Das Axiom „ $\text{isEmpty}(\text{add}(s, c)) = \text{false}$ “ setzt voraus, dass es kein „leeres“ Zeichen geben soll.

Vollständig ? Nein. Zum Beispiel: Die Gleichungen implizieren nicht:

$\text{equal}(s1, s2) = \text{false}$ (für $s1 \# s2$ mit $\text{length}(s1) = \text{length}(s2)$)
obwohl dies intuitiv gelten sollte.

Anmerkung: Kommutativität von equal folgt allerdings schon aus den Folien der vorigen Folie (Beweis per Induktion).

- Spezifikation (lat.): Auflistung
 - Auflistung syntaktisch formulierbarer Eigenschaften einer Algebra
- Signaturen repräsentieren die einfachste Form der Spezifikation
- Idee: Wie kann die Interpretation einer Signatur durch eine Algebra als Funktion dargestellt werden ?
 - Bildung von Formeln über den Termen einer Signatur
- Spezifikationen erweitern also Signaturen um Formeln, die mit Hilfe von Termen syntaktisch formuliert werden
 - Algebraische Gleichungslogik

2.9 Algebraische Spezifikation

Softwarekonstruktion
WS 2016/17

2.9 Algebraische Spezifikation



Das Spezifikationsproblem

Signatur und Algebra

Vollständigkeit der Algebraischen Spezifikation

Algebraische Spezifikation: Diskussion

$\Sigma = (S, F)$ heißt **algebraische Signatur**

- S eine Menge von **Sorten (Typen)**.
- F eine Menge von **Operationssymbolen**.
- Auf F ist eine **Abbildung** definiert: $type: F \rightarrow S^* \times S$
wobei $S^0 = \{ \emptyset \}$ und $S^* = \bigcup_{n \in \mathbb{N}} S^n$
- Für $type(f) = (s_1, \dots, s_n, s)$ schreiben wir $f: s_1, \dots, s_n \rightarrow s$.
- $type(f)$ bezeichnet man auch als **Signatur** der Operation f .
- **Konstanten:** Abbildungen aus F , deren Vorbereich leere Menge ist.

$\Sigma_{\text{Test}} = (\mathbf{S}, \mathbf{F})$ mit

$\mathbf{S} = \{\text{Nat}, \text{Bool}\}$

$\mathbf{F} = \{\text{zero}, \text{one}, \text{succ}, \text{add}, \text{equal}, \text{equalBool}, \text{T}, \text{F}\}$

$\text{zero}: \quad \quad \quad \rightarrow \text{Nat} \quad \quad \quad [\text{d.h. type}(\text{zero}) = (\emptyset, \text{Nat})]$

$\text{one}: \quad \quad \quad \rightarrow \text{Nat}$

$\text{succ}: \quad \quad \text{Nat} \quad \quad \rightarrow \text{Nat}$

$\text{add}: \quad \quad \text{Nat} \times \text{Nat} \quad \rightarrow \text{Nat}$

$\text{equal}: \quad \quad \text{Nat} \times \text{Nat} \quad \rightarrow \text{Bool} \quad [\text{d.h. type}(\text{equal}) = (\text{Nat}, \text{Nat}, \text{Bool})]$

$\text{equalBool}: \quad \text{Bool} \times \text{Bool} \quad \rightarrow \text{Bool}$

$\text{T}: \quad \quad \quad \rightarrow \text{Bool}$

$\text{F}: \quad \quad \quad \rightarrow \text{Bool}$

Interpretiert eine Signatur Σ , indem sie Symbole in Σ mit **konkreten Mengen und Abbildungen** „füllt“.

Σ -Algebra (Definition):

- Sei $\Sigma = (S, F)$ eine Signatur.
- Für alle $s \in S$ sei A_s eine Menge.
- Für alle $f \in F$ mit $f: s_1, \dots, s_n \rightarrow s$ sei $f_A: A_{s_1} \times \dots \times A_{s_n} \rightarrow A_s$ eine Abbildung.
- Dann ist das Paar $A = ((A_s)_{s \in S}, (f_A)_{f \in F})$ eine Σ -Algebra.
- A_s bezeichnet man auch als **Trägermenge** von A zu s , f_A die Interpretation der Operation f .
- Beschreibung der Operation f in Algebra A umfasst ihre Signatur $\text{type}(f)$ und ihre Semantik f_A .
- **Semantik von Operationen: Axiomatisch.**

Signatur Σ_{Test}	Σ_{Test} -Algebra A
Nat	$A_{\text{Nat}} =_{\text{def}} \mathbb{N}$
Bool	$A_{\text{Bool}} =_{\text{def}} \{\text{true}, \text{false}\}$
zero: $\rightarrow \text{Nat}$	$\text{zero}_A =_{\text{def}} 0$
one: $\rightarrow \text{Nat}$	$\text{one}_A =_{\text{def}} 1$
succ: $\text{Nat} \rightarrow \text{Nat}$	$\forall n \in \mathbb{N}. \text{succ}_A(n) =_{\text{def}} n + 1$
add: $\text{Nat} \times \text{Nat} \rightarrow \text{Nat}$	$\forall n, m \in \mathbb{N}. \text{add}_A(n, m) =_{\text{def}} n + m$
equal: $\text{Nat} \times \text{Nat} \rightarrow \text{Bool}$	$\text{equal}_A(\text{zero}_A, \text{zero}_A) =_{\text{def}} \text{true}$ $\forall n, m \in \mathbb{N}. \text{equal}_A(\text{succ}_A(n), \text{succ}_A(m)) =_{\text{def}} \text{equal}_A(n, m)$ $\forall n, m \in \mathbb{N}. \text{equal}_A(n, m) =_{\text{def}} \text{equal}_A(m, n)$ $\forall m \in \mathbb{N}. \text{equal}_A(\text{zero}_A, \text{add}_A(\text{one}_A, m)) =_{\text{def}} \text{false}$

Anmerkung: Axiome definieren eindeutig eine Funktion (müsste genaugenommen jeweils bewiesen werden).

Σ -Algebra

Beispiel (Fortsetzung)

Softwarekonstruktion
WS 2016/17

Signatur Σ_{Test}	Σ_{Test} -Algebra A
$\text{equalBool} : \text{Bool} \times \text{Bool} \rightarrow \text{Bool}$	$\text{equalBool}_A(\text{true}, \text{true}) =_{\text{def}} \text{true}$ $\text{equalBool}_A(\text{false}, \text{false}) =_{\text{def}} \text{true}$ $\text{equalBool}_A(\text{true}, \text{false}) =_{\text{def}} \text{false}$ $\text{equalBool}_A(\text{false}, \text{true}) =_{\text{def}} \text{false}$
$T \rightarrow \text{Bool}$	$T_A =_{\text{def}} \text{true}$
$F \rightarrow \text{Bool}$	$F_A =_{\text{def}} \text{false}$

- Beispiele mit genau **einer Trägermenge**:
 - Σ_{Nat} -Algebra B (A_{Nat} mit den relevanten Operationen und Axiomen aus Σ_{Test}).
 - Gruppen, Ringe, Körper, Verbände.
- Beispiele mit **mehreren Trägermengen**:
 - Σ_{Test} -Algebra A .
 - Vektorräume (Vektor und Skalare als Träger).

Teil 1: **Wichtiges Ziel:** Wiederverwendbarkeit von Software-Komponenten.

Frage: Wie können Ansätze, die auf algebraischer Spezifikation basieren, dieses Ziel unterstützen ?

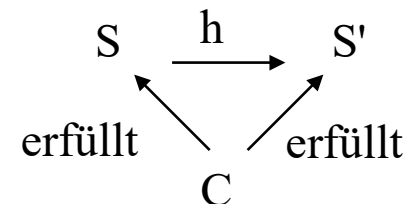
Teil 1: Wichtiges Ziel: Wiederverwendbarkeit von Software-Komponenten.

Frage: Wie können Ansätze, die auf algebraischer Spezifikation basieren, dieses Ziel unterstützen ?

Antwort: Für gegebene algebraische Spezifikation S einer Software-Komponente kann ich eine Alt-Komponente C wiederverwenden, sofern sie Spezifikation S erfüllt.

Frage: Was tun, wenn Alt-Komponente C gegen (etwas) abweichenden algebraischen Spezifikation S' entwickelt wurde ?

Antwort: Definiere Abbildung h zwischen algebraischen Spezifikationen S und S' , die wesentlichen gewünschten Eigenschaften bewahrt (genannt „Homomorphismus“ bzw. „Isomorphismus“).



- $\text{eval}(A)(t)$ wertet alle Grundterme t in einer Algebra A aus
- $\text{xeval}(\text{ass})(t)$ wie eval jedoch mit Variablenbelegung ass
- Für Algebra A und Gleichung $t_l = t_r$ gilt
 - Sind t_l und t_r Grundterme gilt $\text{eval}(A)(t_l) = \text{eval}(A)(t_r)$
 - Besitzen t_l und t_r Variablen gilt Gleichung nur, wenn für alle passenden Variablenbelegungen ass gilt: (Extensionalität)
$$\text{xeval}(\text{ass})(t_l) = \text{xeval}(\text{ass})(t_r)$$

Definition: Σ -Gleichung / Grundgleichung

- Sei A eine Algebra mit der Signatur $\Sigma = (S, F, X)$ und $s \in S$.
Für $t, t' \in T(\Sigma, s)(X)$ heißt
 $t =_s t'$ eine **Σ -Gleichung**.
- $t =_s t'$ ist „gültig“ in einer Algebra A , falls für alle Variablenbelegungen die Auswertung der Terme identisch ist.
- $t =_s t'$ heißt **Grundgleichung**, falls weder t noch t' Variablen enthalten.

Beispiele:

- Signatur Σ -Nat, Σ -Algebra A
 - $\text{eval}(A)_{\text{Nat}}(\text{zero}) = \text{eval}(A)_{\text{Nat}}(\text{add}(\text{zero}, \text{zero}))$?
 - $\text{eval}(A)_{\text{Nat}}(\text{zero}) = \text{eval}(A)_{\text{Nat}}(\text{one})$?
 - $\text{xeval}(\text{ass})_{\text{Nat}}(\text{add}(n_1, n_2)) = \text{xeval}(\text{ass})_{\text{Nat}}(\text{add}(n_2, n_1))$?

Definition: Σ -Gleichung / Grundgleichung

- Sei A eine Algebra mit der Signatur $\Sigma = (S, F, X)$ und $s \in S$.
Für $t, t' \in T(\Sigma, s)(X)$ heißt
 $t =_s t'$ eine **Σ -Gleichung**.
- $t =_s t'$ ist „gültig“ in einer Algebra A , falls für alle Variablenbelegungen die Auswertung der Terme identisch ist.
- $t =_s t'$ heißt **Grundgleichung**, falls weder t noch t' Variablen enthalten.

Beispiele:

- Signatur Σ -Nat, Σ -Algebra A
 - $\text{eval}(A)_{\text{Nat}}(\text{zero}) = \text{eval}(A)_{\text{Nat}}(\text{add}(\text{zero}, \text{zero}))$ gültig
 - $\text{eval}(A)_{\text{Nat}}(\text{zero}) = \text{eval}(A)_{\text{Nat}}(\text{one})$?
 - $\text{xeval}(\text{ass})_{\text{Nat}}(\text{add}(n_1, n_2)) = \text{xeval}(\text{ass})_{\text{Nat}}(\text{add}(n_2, n_1))$?

Definition: Σ -Gleichung / Grundgleichung

- Sei A eine Algebra mit der Signatur $\Sigma = (S, F, X)$ und $s \in S$.
Für $t, t' \in T(\Sigma, s)(X)$ heißt
 $t =_s t'$ eine **Σ -Gleichung**.
- $t =_s t'$ ist „gültig“ in einer Algebra A , falls für alle Variablenbelegungen die Auswertung der Terme identisch ist.
- $t =_s t'$ heißt **Grundgleichung**, falls weder t noch t' Variablen enthalten.

Beispiele:

- Signatur Σ -Nat, Σ -Algebra A
 - $\text{eval}(A)_{\text{Nat}}(\text{zero}) = \text{eval}(A)_{\text{Nat}}(\text{add}(\text{zero}, \text{zero}))$ gültig
 - $\text{eval}(A)_{\text{Nat}}(\text{zero}) = \text{eval}(A)_{\text{Nat}}(\text{one})$ nicht gültig
 - $\text{xeval}(\text{ass})_{\text{Nat}}(\text{add}(n_1, n_2)) = \text{xeval}(\text{ass})_{\text{Nat}}(\text{add}(n_2, n_1))$?

Definition: Σ -Gleichung / Grundgleichung

- Sei A eine Algebra mit der Signatur $\Sigma = (S, F, X)$ und $s \in S$.
Für $t, t' \in T(\Sigma, s)(X)$ heißt
 $t =_s t'$ eine **Σ -Gleichung**.
- $t =_s t'$ ist „gültig“ in einer Algebra A , falls für alle Variablenbelegungen die Auswertung der Terme identisch ist.
- $t =_s t'$ heißt **Grundgleichung**, falls weder t noch t' Variablen enthalten.

Beispiele:

- Signatur Σ -Nat, Σ -Algebra A
 - $\text{eval}(A)_{\text{Nat}}(\text{zero}) = \text{eval}(A)_{\text{Nat}}(\text{add}(\text{zero}, \text{zero}))$ gültig
 - $\text{eval}(A)_{\text{Nat}}(\text{zero}) = \text{eval}(A)_{\text{Nat}}(\text{one})$ nicht gültig
 - $\text{xeval}(\text{ass})_{\text{Nat}}(\text{add}(n_1, n_2)) = \text{xeval}(\text{ass})_{\text{Nat}}(\text{add}(n_2, n_1))$ gültig

Definition: Menge der wohldefinierten Formeln

Definition: Menge der wohldefinierten Formeln

Zu einer Algebra mit der Signatur $\Sigma = (S, F, X)$ und einer S-sortierten Variablenmenge X wird die **Menge der wohldefinierten Formeln** über der Signatur Σ definiert als die Menge $WFF(\Sigma)$, die die kleinste Menge mit den folgenden Eigenschaften ist:

- Jede Σ -Gleichung ist in $WFF(\Sigma)$.
- Für alle Formeln G und H in $WFF(\Sigma)$ sind auch die Formeln $\neg G$, $(G \wedge H)$ und $(G \vee H)$ enthalten in $WFF(\Sigma)$.

Bemerkung: Man könnte auch die $\forall$ - und $\exists$ -Quantoren einbeziehen; zur Vereinfachung verzichten wir hier darauf.

Frage: Was ist mit $G \Rightarrow H$?

Definition: Menge der wohldefinierten Formeln

Definition: Menge der wohldefinierten Formeln

Zu einer Algebra mit der Signatur $\Sigma = (S, F, X)$ und einer S-sortierten Variablenmenge X wird die **Menge der wohldefinierten Formeln** über der Signatur Σ definiert als die Menge $WFF(\Sigma)$, die die kleinste Menge mit den folgenden Eigenschaften ist:

- Jede Σ -Gleichung ist in $WFF(\Sigma)$.
- Für alle Formeln G und H in $WFF(\Sigma)$ sind auch die Formeln $\neg G$, $(G \wedge H)$ und $(G \vee H)$ enthalten in $WFF(\Sigma)$.

Bemerkung: Man könnte auch die $\forall$ - und $\exists$ -Quantoren einbeziehen; zur Vereinfachung verzichten wir hier darauf.

Frage: Was ist mit $G \Rightarrow H$?

Antwort: $G \Rightarrow H$ ist äquivalent mit $\neg G \vee H$.

Menge der wohldefinierten Formeln: Beispiel

Zur Signatur Σ_{Nat} enthält $\text{WFF}(\Sigma_{\text{Nat}})$ beispielsweise:

- jede Σ_{Nat} -Gleichung aus $T(\Sigma_{\text{Nat}}, S)$, z.B. $\text{add}(\text{zero}, n) = n$
- Negationen, Disjunktion und Konjunktionen,
z.B. $\neg (\text{add}(\text{zero}, n) = n)$

Definition: Algebraische Spezifikation

Sei $\Sigma = (S, F, X)$ eine Signatur mit Variablen und E eine Menge von Σ -Formeln mit $E \subseteq \text{WFF}(\Sigma)$. Dann heißt $SP = \langle \Sigma, E \rangle$ **algebraische Spezifikation**.

Beispiel:

Die algebraische Spezifikation Spec-Nat erweitert die Signatur Σ_{Nat} um die Σ_{Nat} -Formeln (in diesem Fall alle Gleichungen):

- $\text{equal}_{\text{Nat}}(\text{zero}, \text{zero}) = \text{true}$
- $\text{equal}_{\text{Nat}}(\text{succ}_{\text{Nat}}(n_1), \text{succ}_{\text{Nat}}(n_2)) = \text{equal}_{\text{Nat}}(n_1, n_2)$
- $\text{succ}_{\text{Nat}}(n_1) = \text{add}_{\text{Nat}}(n_1, 1)$
- $\text{succ}_{\text{Nat}}(\text{zero}) = \text{one}$

Sei $SP = \langle \Sigma, E \rangle$ eine algebraische Spezifikation und A eine Σ -Algebra.

Dann heißt A **Modell** von SP (auch genannte eine „SP-Algebra“), wenn alle Formeln aus E in A erfüllt sind, d.h.:

- für jede Gleichung $t = t'$ aus E und jede Variablenauswertung $\text{ass}: X \rightarrow A$ gilt: $\text{xeval}(\text{ass})(t) = \text{xeval}(\text{ass})(t')$ und
- analog gelten alle Formeln $\neg G$, $(G \wedge H)$ und $(G \vee H)$ aus E in A für jede Variablenauswertung $\text{ass}: X \rightarrow A$.

Modelle sind also Algebren mit der gegebenen Signatur, die die gegebenen Formeln erfüllen.

Die bereits definierte Algebra A ist ein Modell der algebraischen Spezifikation $\text{Spec-Nat} = (\sum_{\text{Nat}}, E)$, wenn A alle Formeln aus E erfüllt.

Beispielhafte Überprüfung anhand des Beispielaxioms

$\text{equal}_{\text{Nat}}(\text{succ}_{\text{Nat}}(n_1), \text{succ}_{\text{Nat}}(n_2)) = \text{equal}_{\text{Nat}}(n_1, n_2)$:

$$\begin{aligned} & xeval(v)_{\text{Nat}}(\text{equal}(\text{suc}(n_1), \text{suc}(n_2))) = xeval(v)_{\text{Nat}}(\text{equal}(n_1, n_2)) \\ & \Leftrightarrow \text{equal}_A(xeval(v)_{\text{Nat}}(\text{succ}(n_1)), xeval(v)_{\text{Nat}}(\text{succ}(n_2))) \\ & \quad = \text{equal}_A(xeval(v)_{\text{Nat}}(n_1), xeval(v)_{\text{Nat}}(n_2)) \\ & \Leftrightarrow \text{equal}_A(\text{succ}_A(v_{\text{Nat}}(n_1)), \text{succ}_A(v_{\text{Nat}}(n_2))) = \text{equal}_A(v_{\text{Nat}}(n_1), v_{\text{Nat}}(n_1)) \end{aligned}$$

Letztere Gleichung gilt für jede Variablenauswertung v_{Nat} , weil zwei natürliche Zahlen i, j genau dann gleich sind, wenn $i+1=j+1$.

Im allgemeinen kann eine algebraische Spezifikation mehrere Modelle haben. Besonders geeignet sind Modelle, die die folgenden beiden Eigenschaften erfüllen:

- Jedes Element des Modells kann mittels Termen der Signatur bezeichnet werden (**no-junk-Prinzip**).
 - Eine Algebra, die Elemente in ihren Trägermengen hat, die wir mit Grundtermen nicht bezeichnen können, war mit der Signatur vermutlich nicht „gemeint“ (sonst wäre die Signatur entsprechend größer gewählt).
 - Alles was von der Signatur (mittels der Grundterme) nicht erfasst wird, wird als „junk“ bezeichnet.
- In dem Modell gelten keine Gleichungen, die nicht zwingend aus den Axiomen folgen würden (**no-confusion-Prinzip**).
 - Das heißt, wir interpretieren die durch eine Signatur gegebene Spezifikation minimal. Sie beschreibt nichts „Überflüssiges“. Wäre eine Algebra „gemeint“ gewesen, in der weitere Gleichungen gelten, die nicht aus den Axiomen folgen, hätte diese in den Axiomen spezifiziert werden sollen.

Gute Nachricht: Für jede algebraische Spezifikation kann mittels der Grundterme und der gegebenen Axiome ein Modell konstruiert werden (das sogenannte „initiale Modell“), dass die o.g. Eigenschaften hat (lassen wir hier aus Zeitgründen aus).

Beispiel: Junk / confusion

Σ_{NatB} (ohne +)	Nat	INT	NATMOD3
Nat	N	Z	{0,1,2}
zero: $\rightarrow \text{Nat}$ succ: $\text{Nat} \rightarrow \text{Nat}$	0 $n \rightarrow n+1$	0 $i \rightarrow i+1$	0 $n \rightarrow (n+1)\text{mod}3$
(keine Axiome)	?	?	?

- Die Standardalgebra Nat enthält ...

Σ_{NatB} (ohne +)	Nat	INT	NATMOD3
Nat	N	Z	{0,1,2}
zero: $\rightarrow$ Nat succ: Nat $\rightarrow$ Nat	0 $n \rightarrow n+1$	0 $i \rightarrow i+1$	0 $n \rightarrow (n+1)\text{mod}3$
(keine Axiome)	No junk / confusion	?	?

- Die Standardalgebra Nat enthält weder „junk“ noch „confusion“.

Σ_{NatB} (ohne +)	Nat	INT	NATMOD3
Nat	N	Z	{0,1,2}
zero: $\rightarrow \text{Nat}$ succ: $\text{Nat} \rightarrow \text{Nat}$	0 $n \rightarrow n+1$	0 $i \rightarrow i+1$	0 $n \rightarrow (n+1)\text{mod}3$
(keine Axiome)	No junk / confusion	Junk	?

- Die Standardalgebra Nat enthält weder „junk“ noch „confusion“.
- INT enthält „junk“, da nicht alle Elemente der Trägermenge $\text{INT}_{\text{NAT}} = \mathbb{Z}$ von Grundtermen bezeichnet werden können.
Alle negativen Zahlen bleiben syntaktisch unbenannt.

Σ_{NatB} (ohne +)	Nat	INT	NATMOD3
Nat	N	Z	{0,1,2}
zero: $\rightarrow$ Nat succ: Nat $\rightarrow$ Nat	0 $n \rightarrow n+1$	0 $i \rightarrow i+1$	0 $n \rightarrow (n+1)\text{mod}3$
(keine Axiome)	No junk / confusion	Junk	Confusion

- Die Standardalgebra Nat enthält weder „junk“ noch „confusion“.
- INT enthält „junk“, da nicht alle Elemente der Trägermenge $\text{INT}_{\text{NAT}} = \mathbb{Z}$ von Grundtermen bezeichnet werden können.
Alle negativen Zahlen bleiben syntaktisch unbenannt.
- NATMOD3 enthält keinen „junk“, ist aber „confus“, denn es gelten Gleichungen (z.B. $\text{zero}() = \text{succ}(\text{succ}(\text{succ}(\text{zero()})))$), die nicht aus den Axiomen folgen.

Definition: Eine algebraische Spezifikation ist **widersprüchlich**, wenn sich $\text{true} = \text{false}$ beweisen lässt.

- **Beispiel:** Eine Spezifikation, die die Axiome $c = c'$ und $\neg(c = c')$ für zwei Konstanten c, c' enthält.

Definition: Eine algebraische Spezifikation ist redundant, wenn die Menge der beweisbaren Aussagen gleich bleibt auch dann, wenn ein Axiom gestrichen wird.

- **Beispiel:** Wenn die „String“-Algebra um das Axiom $\text{append}(\text{new}(), s) = s$ erweitert wird, dann ist sie redundant, denn dieses Axiom ist ohnehin ableitbar und trägt somit nicht zu zusätzlichen beweisbaren Axiomen bei.

Beide Eigenschaften sind unerwünscht: Aus einer widersprüchlichen Spezifikation lässt sich jede Aussage ableiten (sie ist somit inhaltsleer), eine redundante Spezifikation verursacht unnötig viel Aufwand bei der Anwendung durch die überflüssigen Axiome.

2.9 Algebraische Spezifikation

Softwarekonstruktion
WS 2016/17

2.9 Algebraische Spezifikation



Das Spezifikationsproblem

Signatur und Algebra

Vollständigkeit der Algebraischen Spezifikation

Algebraische Spezifikation: Diskussion

algebra StringSpec

introduces sorts String, Char, Nat, Bool

operations

new: $\rightarrow$ String

append: String, String $\rightarrow$ String

add: String, Char $\rightarrow$ String

length: String $\rightarrow$ Nat

isEmpty: String $\rightarrow$ Bool

equal: String, String $\rightarrow$ Bool

Spezifikation des Datentyps String

algebra StringSpec **introduces sorts** String, Char, Nat, Bool

operations

constraints new, append, add, length, isEmpty, equal

so that for all s,s1,s2: String, c: Char

$\text{isEmpty}(\text{new}()) = \text{true}$

$\text{isEmpty}(\text{add}(s,c)) = \text{false}$

$\text{length}(\text{new}()) = 0$

$\text{length}(\text{add}(s,c)) = \text{length}(s) + 1$

$\text{append}(s, \text{new}()) = s$

$\text{append}(s1, \text{add}(s2,c)) = \text{add}(\text{append}(s1,s2),c)$

$\text{equal}(\text{new}(), \text{new}()) = \text{true}$

$\text{equal}(\text{new}(), \text{add}(s,c)) = \text{false}$

$\text{equal}(\text{add}(s,c), \text{new}()) = \text{false}$

$\text{equal}(\text{add}(s1,c), \text{add}(s2,c)) = \text{equal}(s1,s2)$

Frage:

Gilt

(i) $\text{append}(\text{new}(), \text{add}(\text{new}(), c)) = \text{add}(\text{new}(), c)$??

Frage:

Gilt

(i) $\text{append}(\text{new}(), \text{add}(\text{new}(), c)) = \text{add}(\text{new}(), c)$??

Start vom Axiom:

(ii) $\text{append}(s1, \text{add}(s2, c)) = \text{add}(\text{append}(s1, s2), c)$

Frage:

Gilt

$$(i) \text{ append (new(), add (new(), c)) = add (new(), c) } \quad ??$$

Start vom Axiom:

$$(ii) \text{ append (s1, add(s2,c)) = add (append (s1,s2),c)}$$

Wir ersetzen hier s1 und s2 durch new():

$$(iii) \text{ append (new(), add (new(),c)) = add (append (new(), new()),c)}$$

Frage:

Gilt

$$(i) \text{ append (new(), add (new(), c)) = add (new(), c) } \quad ??$$

Start vom Axiom:

$$(ii) \text{ append (s1, add(s2,c)) = add (append (s1,s2),c)}$$

Wir ersetzen hier s1 und s2 durch new():

$$(iii) \text{ append (new(), add (new(),c)) = add (append (new(), new()),c)}$$

Im Axiom $\text{append (s, new()) = s}$ ersetzen wir s durch new() und erhalten:

$$(iv) \text{ append (new(), new()) = new() }$$

Frage:

Gilt

$$(i) \text{ append (new(), add (new(), c)) = add (new(), c) } \quad ??$$

Start vom Axiom:

$$(ii) \text{ append (s1, add(s2,c)) = add (append (s1,s2),c)}$$

Wir ersetzen hier s1 und s2 durch new():

$$(iii) \text{ append (new(), add (new(),c)) = add (append (new(), new()),c)}$$

Im Axiom $\text{append (s, new()) = s}$ ersetzen wir s durch new() und erhalten:

$$(iv) \text{ append (new(), new()) = new()}$$

In (iii) setzen wir (iv) ein und erhalten:

$$(v) \text{ append (new(), add (new(),c)) = add (new(),c)}$$

womit (i) gezeigt ist.

Spezifikation des Datentyps String

Vollständigkeit der Axiome II

Softwarekonstruktion
WS 2016/17



Frage: Gilt (i) $\text{append}(\text{new}(), s) = s$??

Frage: Gilt (i) $\text{append}(\text{new}(), s) = s$??

- Induktionsanfang

Offensichtlich gilt (i) für $s = \text{new}()$, denn $\text{append}(\text{new}(), \text{new}()) = \text{new}()$ gilt nach Axiom $\text{append}(s, \text{new}()) = s$

Frage: Gilt (i) $\text{append}(\text{new}(), s) = s$??

- Induktionsanfang

Offensichtlich gilt (i) für $s = \text{new}()$, denn $\text{append}(\text{new}(), \text{new}()) = \text{new}()$ gilt nach Axiom $\text{append}(s, \text{new}()) = s$

- Induktionsannahme:

Wir wählen ein beliebiges s' sodass $s = \text{add}(s', c)$ und nehmen an, dass (i) für s' gilt.

Frage: Gilt (i) $\text{append}(\text{new}(), s) = s$??

- Induktionsanfang

Offensichtlich gilt (i) für $s = \text{new}()$, denn $\text{append}(\text{new}(), \text{new}()) = \text{new}()$ gilt nach Axiom $\text{append}(s, \text{new}()) = s$

- Induktionsannahme:

Wir wählen ein beliebiges s' sodass $s = \text{add}(s', c)$ und nehmen an, dass (i) für s' gilt.

- Nachweis des Axioms

Damit gilt:

$$\begin{aligned}\text{append}(\text{new}(), s) &= \\ \text{append}(\text{new}(), \text{add}(s', c)) &= \\ \text{add}(\text{append}(\text{new}(), s'), c) &= \\ \text{add}(s', c) &= \\ s\end{aligned}$$

Induktionsannahme

$$\begin{aligned}\text{Axiom } \text{append}(s1, \text{add}(s2, c)) \\ = \text{add}(\text{append}(s1, s2), c)\end{aligned}$$

Induktionsannahme

Induktionsannahme

Frage: Gilt (i) $\text{append}(\text{new}(), s) = s$??

- Induktionsanfang

Offensichtlich gilt (i) für $s = \text{new}()$, denn $\text{append}(\text{new}(), \text{new}()) = \text{new}()$ gilt nach Axiom $\text{append}(s, \text{new}()) = s$

- Induktionsannahme:

Wir wählen ein beliebiges s' sodass $s = \text{add}(s', c)$ und nehmen an, dass (i) für s' gilt.

- Nachweis des Axioms

Damit gilt:

$\text{append}(\text{new}(), s) =$

Induktionsannahme

$\text{append}(\text{new}(), \text{add}(s', c)) =$

Axiom $\text{append}(s1, \text{add}(s2, c))$
 $= \text{add}(\text{append}(s1, s2), c)$

$\text{add}(\text{append}(\text{new}(), s'), c) =$

Induktionsannahme

$\text{add}(s', c) =$

Induktionsannahme

s

Frage: Für welche Elemente in String beweist Induktionsbeweis o.g. Gleichung ?

Frage: Gilt (i) $\text{append}(\text{new}(), s) = s$??

- Induktionsanfang

Offensichtlich gilt (i) für $s = \text{new}()$, denn $\text{append}(\text{new}(), \text{new}()) = \text{new}()$ gilt nach Axiom $\text{append}(s, \text{new}()) = s$

- Induktionsannahme:

Wir wählen ein beliebiges s' sodass $s = \text{add}(s', c)$ und nehmen an, dass (i) für s' gilt.

- Nachweis des Axioms

Damit gilt:

$\text{append}(\text{new}(), s) =$

Induktionsannahme

$\text{append}(\text{new}(), \text{add}(s', c)) =$

Axiom $\text{append}(s1, \text{add}(s2, c))$
 $= \text{add}(\text{append}(s1, s2), c)$

$\text{add}(\text{append}(\text{new}(), s'), c) =$

Induktionsannahme

$\text{add}(s', c) =$

Induktionsannahme

s

Frage: Für welche Elemente in String beweist Induktionsbeweis o.g. Gleichung ?

Antwort: Strings, die durch new und add erzeugt werden.

- Letzter Beweis basiert auf **Annahme**: Alle Strings sind durch Operationen *new* und *add* erzeugbar (nur für diese gilt der Beweis).
- **Identifikation erzeugender Menge** von Operationen. →
Zur Ermittlung erforderlicher Axiome und Sicherstellung der Beweiskraft von Induktionsbeweisen.
- **Definition**: Menge O von Operationen heißt „**generierend**“ für Menge X , wenn alle Elemente in X durch sukzessive Anwendung der Operationen erzeugt werden können.
- **Bemerkung**: Insbesondere kann Menge O null-stellige Operationen (= Konstanten) enthalten.

Frage:

- Was ist generierende Menge von Operationen der Booleschen Algebra ?
- Was ist generierende Menge von Operationen der Algebra der positiven, ganzen Zahlen ?

Frage:

- Was ist generierende Menge von Operationen der Booleschen Algebra ?

Antwort: zum Beispiel {false, not}

- Was ist generierende Menge von Operationen der Algebra der positiven, ganzen Zahlen ?

Frage:

- Was ist generierende Menge von Operationen der Booleschen Algebra ?

Antwort: zum Beispiel {false, not}

- Was ist generierende Menge von Operationen der Algebra der positiven, ganzen Zahlen ?

Antwort: zum Beispiel {zero, succ}

Im Rahmen algebraischer Spezifikation können wir definieren, dass Typen von bestimmten Operationen generiert werden:

constraints *<operations>* **so that**
 <Type> **generated by** [*<generating ops>*]
for all *<variables>*: *<Type>* ...

Beispiel:

constraints new, append, add, length, isEmpty, equal **so that**
 String **generated by** [new,add,Char]
for all s,s1,s2: String, c: Char

Bemerkung: Hier wird Char als Menge von Atomen (generierende Konstanten) vorausgesetzt.

Erweiterung algebraischer Spezifikation *String* um Konstanten 'a', 'b' von Typ Char (formal: nullstellige Operation $a: () \rightarrow 'a'$).

Frage:

```
isEmpty(new()) = true
isEmpty(add (s,c)) = false
length (new()) = 0
length (add(s,c)) = length(s) + 1
append (s, new()) = s
append (s1, add(s2,c)) = add (append(s1,s2),c)
equal (new(), new()) = true
equal (new(), add(s,c)) = false
equal (add(s,c), new()) = false
equal (add(s1,c), add(s2,c)) = equal (s1,s2)
```

$\text{equal} (\text{add} (s, 'a'), \text{add} (s, 'b')) = \text{false} ?$

Erweiterung der algebraischen Spezifikation *String* um Konstanten 'a', 'b' von Typ Char (formal: nullstellige Operation $a: () \rightarrow 'a'$).

Frage:

```
isEmpty(new()) = true
isEmpty(add (s,c)) = false
length (new()) = 0
length (add(s,c)) = length(s) + 1
append (s, new()) = s
append (s1, add(s2,c)) = add (append(s1,s2),c)
equal (new(), new()) = true
equal (new(), add(s,c)) = false
equal (add(s,c), new()) = false
equal (add(s1,c), add(s2,c)) = equal (s1,s2)
```

$equal (add (s, 'a'), add (s, 'b')) = false$?

- Sollte intuitiv gelten, lässt sich aber nicht aus Axiomen ableiten.

→ **Algebraische Spezifikation: Unvollständig**, weil nicht alle (intuitiv) als wahr angenommene Aussagen beweisbar.

Vervollständigung von *String* durch zusätzliche Operation

$equalC : Char, Char \rightarrow Bool$

mit den Axiomen:

$$equalC ('a', 'a') = true$$

$$equalC ('a', 'b') = false$$

$$equalC ('b', 'a') = false$$

$$equalC ('b', 'b') = true$$

und der Ersetzung des Axioms

$$equal (add (s1, c), add (s2, c)) = equal (s1, s2)$$

durch

$$equal (add (s1, c1), add (s2, c2)) = equal (s1, s2) \wedge equalC (c1, c2)$$

Frage:

*equal (add (s,'a'), add (s,'b'))
= false ?*

```
isEmpty(new()) = true
isEmpty(add (s,c)) = false
length (new()) = 0
length (add(s,c)) = length(s) + 1
append (s, new()) = s
append (s1, add(s2,c)) = add (append(s1,s2),c)
equal (new(), new()) = true
equal (new(), add(s,c)) = false
equal (add(s,c), new()) = false

equal (add(s1,c), add(s2,c)) = equal (s1,s2)

equalC ('a','a') = true      (usw...)
equalC ('a','b') = false
equal (add (s1,c1), add (s2,c2))
      = equal (s1,s2)  $\wedge$  equalC (c1,c2)
```

Frage:

*equal (add (s,'a'), add (s,'b'))
= false ?*

Antwort:

equal (add (s,'a'), add (s,'b'))

```
isEmpty(new()) = true
isEmpty(add (s,c)) = false
length (new()) = 0
length (add(s,c)) = length(s) + 1
append (s, new()) = s
append (s1, add(s2,c)) = add (append(s1,s2),c)
equal (new(), new()) = true
equal (new(), add(s,c)) = false
equal (add(s,c), new()) = false

equal (add(s1,c), add(s2,c)) = equal (s1,s2)

equalC ('a','a') = true      (usw...)
equalC ('a','b') = false
equal (add (s1,c1), add (s2,c2))
    = equal (s1,s2) ∧ equalC (c1,c2)
```

Frage:

$\text{equal}(\text{add}(s, 'a'), \text{add}(s, 'b'))$
 $= \text{false} ?$

Antwort:

$\text{equal}(\text{add}(s, 'a'), \text{add}(s, 'b'))$
 $= \text{equal}(s, s) \wedge \text{equalC}('a', 'b')$

$\text{isEmpty}(\text{new}()) = \text{true}$
 $\text{isEmpty}(\text{add}(s, c)) = \text{false}$
 $\text{length}(\text{new}()) = 0$
 $\text{length}(\text{add}(s, c)) = \text{length}(s) + 1$
 $\text{append}(s, \text{new}()) = s$
 $\text{append}(s1, \text{add}(s2, c)) = \text{add}(\text{append}(s1, s2), c)$
 $\text{equal}(\text{new}(), \text{new}()) = \text{true}$
 $\text{equal}(\text{new}(), \text{add}(s, c)) = \text{false}$
 $\text{equal}(\text{add}(s, c), \text{new}()) = \text{false}$

 ~~$\text{equal}(\text{add}(s1, c), \text{add}(s2, c)) = \text{equal}(s1, s2)$~~

 $\text{equalC}('a', 'a') = \text{true} \quad (\text{usw...})$
 $\text{equalC}('a', 'b') = \text{false}$
 $\text{equal}(\text{add}(s1, c1), \text{add}(s2, c2))$
 $\quad = \text{equal}(s1, s2) \wedge \text{equalC}(c1, c2)$

Frage:

$\text{equal}(\text{add}(s, 'a'), \text{add}(s, 'b'))$
 $= \text{false} ?$

Antwort:

$\text{equal}(\text{add}(s, 'a'), \text{add}(s, 'b'))$
 $= \text{equal}(s, s) \wedge \text{equalC}('a', 'b')$
 $= \text{true} \wedge \text{false} = \text{false}$

```
isEmpty(new()) = true
isEmpty(add(s,c)) = false
length(new()) = 0
length(add(s,c)) = length(s) + 1
append(s, new()) = s
append(s1, add(s2,c)) = add(append(s1,s2),c)
equal(new(), new()) = true
equal(new(), add(s,c)) = false
equal(add(s,c), new()) = false

equal(add(s1,c), add(s2,c)) = equal(s1,s2)

equalC('a','a') = true      (usw...)
equalC('a','b') = false
equal(add(s1,c1), add(s2,c2))
    = equal(s1,s2)  $\wedge$  equalC(c1,c2)
```

2.9 Algebraische Spezifikation

2.9 Algebraische Spezifikation



Das Spezifikationsproblem

Signatur und Algebra

Vollständigkeit der Algebraischen Spezifikation

Algebraische Spezifikation: Diskussion

Algebraische Signatur Σ

- Sorten S
- Operationssymbole F
 - Operationssymbole bilden abstrakt von Sorten auf Sorten ab
 - Konstanten bilden auf Sorten ab.
- ggf. Variablen X

Term t

- Verknüpfungen von Konstanten und Operationen ($\rightarrow$ Grundterme) sowie ggf. Variablen ($\rightarrow$ Allgemeine Terme)
- Entsprechend den Regeln der Algebra

algebraische Spezifikation SP

- Signatur Σ und wohldefinierte Formeln E (Gleichungen und ggf. logische Verknüpfungen)

Algebra

- Signatur Σ
- Mengen A
 - Konkrete Ausprägungen der Sorten
- Operationen f
 - konkrete Definition der Operationssymbole

Gleichung e

- Verknüpfung von zwei Termen
- Gültig, wenn beide Terme (ggf. für alle Variablenbelegungen) identisch ausgewertet werden

Modell

- algebraische Spezifikation SP und Algebra A, mit „gültigen“ Gleichungen.

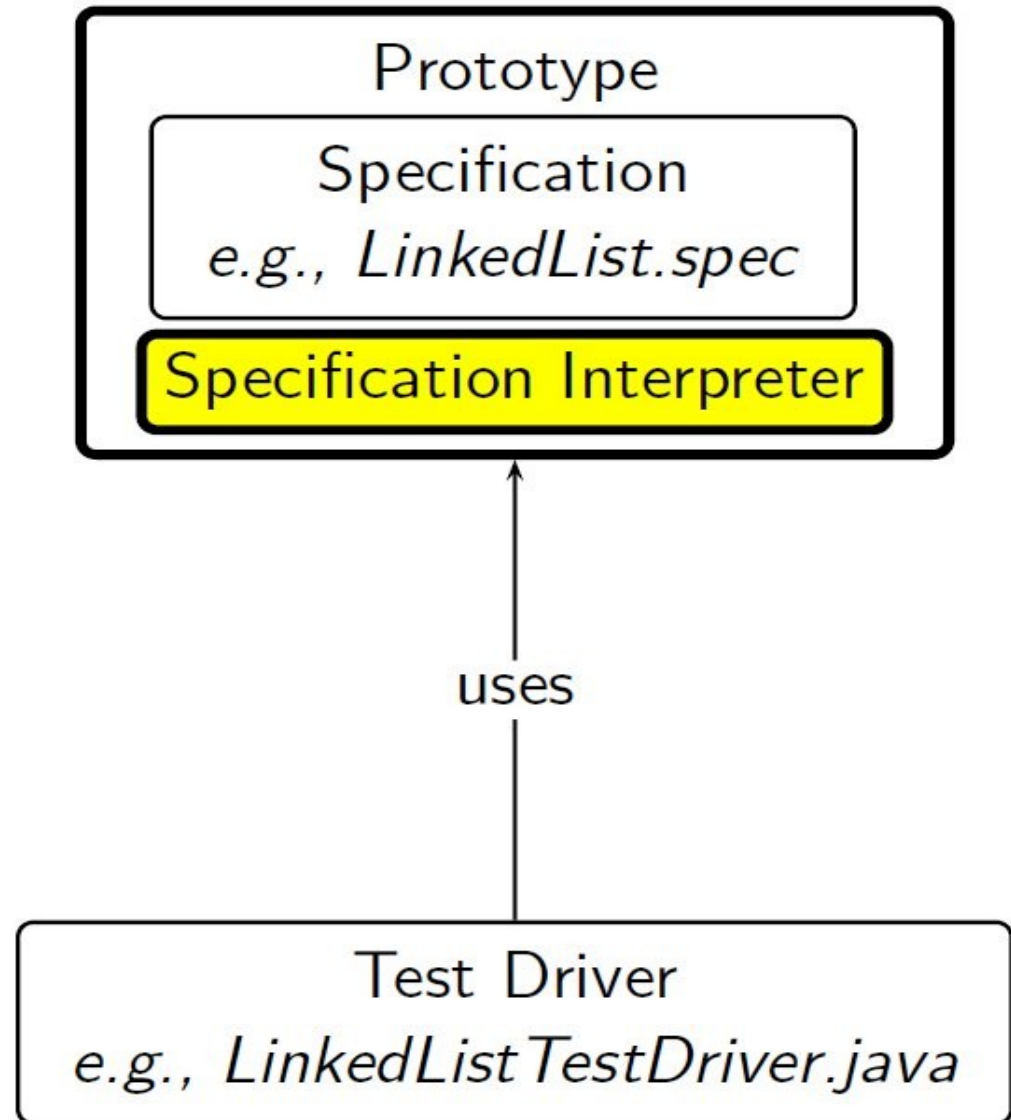
Algebraische Spezifikation:

- **Verwendung in speziellen Einsatzgebieten** (z.B. Einsatz von Kryptographie in Software).
- **Grundlage für „benutzerfreundlichere“ Ansätze**, die allgemein verwendet werden (z.B. Object Constraint Language (OCL) im Rahmen der UML, oder Java / C assertions). [OCL: Teil 3.0 dieser Vorlesung]
- Grundlage für **allgemein verwendete Software-Entwicklungs-Prinzipien** wie „Design-by-contract“ (vgl. Programmiersprache Eiffel oder Java Markup Language (JML)) oder Software-Verifikationswerkzeuge wie VCC. [<http://research.microsoft.com/en-us/projects/vcc>]

Johannes Henkel and Amer
Diwan: A Tool for Writing and
Debugging Algebraic Specifications,
International Conference on
Software Engineering (ICSE) 2004

(<http://www-plan.cs.colorado.edu/henkel/pubs.html>)

Zum Beispiel für **spezifikations-**
basiertes Testen.



Beispiel: Von Java zu algebraischer Spezifikation

```
class LinkedList {  
    Object get(int index){...}  
}
```

[HenDiw04]

Algebraic Signature of get?

$LinkedList \times int \rightarrow \textcolor{red}{LinkedList} \times \textcolor{blue}{Object}$

Receiver object 
Parameter 


Return value
Updated receiver object

Retrieving the 5th element of LinkedList ℓ :

$\text{get}(\ell, 5).\text{state}$ — updated state of ℓ after executing $\ell.\text{get}(5)$

$\text{get}(\ell, 5).\text{retval}$ — return value of $\ell.\text{get}(5)$

Beispiel: Von Java zu algebraischer Spezifikation

[HenDiw04]

original client

algebraic term

```
s = new IntStack();
```

state : *NewIntStack().state*
retval :

```
s.push(5);
```

state : *push(NewIntStack().state, 5).state*
retval :

```
s.push(7);
```

state : *push(push(..., 5).state, 7).state*
retval :

```
i = s.pop();
```

state : *pop(push(..., 7).state).state*
retval : *pop(push(..., 7).state).retval*

- **ASTOOT** [Doong, Frankl 1994]:
<http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.23.2625>
- **Black/White** [Chen et al. 1998]:
<http://citeseer.ist.psu.edu/viewdoc/summary?doi=10.1.1.29.541>
- **TACCLE** [Chen, Tse, Chen 2001]:
<http://citeseer.ist.psu.edu/viewdoc/summary?doi=10.1.1.14.2484>
- **Korat** [Boyapati, Kurshid, Marinov 2002]:
<http://citeseer.ist.psu.edu/viewdoc/summary?doi=10.1.1.122.9788>

- **In diesem Kapitel:**
 - Spezifikationsproblem.
 - Vollständigkeit der Algebra.
 - Signatur und Algebra.
 - Algebraische Spezifikation.
- **Betrachtung:**
 - algebraischer Spezifikation als **Grundlage für Constraint-Spezifikationsansätze** wie OCL.
 - Stellt **grundlegende Konzepte für Spezifikation** des Verhaltens einzelner Softwaremodule bereit.
 - einiger **Beispiele** für praktische Anwendungen dieses Ansatz.

- Ehrig, Mahr: Fundamentals of Algebraic Specification 1: Equations and Initial Semantics, Springer Verlag, 1985
- Ehrig, Mahr: Fundamentals of Algebraic Specification 2: Module Specifications and Constraints, Springer Verlag, 1990
- Ehrig, Mahr, Cornelius et.al.: Mathematisch-strukturelle Grundlagen der Informatik, Springer Verlag, 1999
- Ehrich, Gogolla, Lipeck: Algebraische Spezifikation abstrakter Datentypen, Teubner Verlag, 1989
- Sommerville, Ian: Software Engineering, Addison-Wesley Longman, 2010. Kapitel 27: Formal Specification.
http://ifs.host.cs.st-andrews.ac.uk/Books/SE9/WebChapters/PDF/Ch_27_Formal_spec.pdf